# HackerOne Product Security Analyst

Complete Technical Question & Answer Bank

Full, in-depth answers to every technical question from the interview prep guide — OWASP Top 10, HackerOne triage methodology, CVSS scoring, mobile security, open-source-specific reasoning, and report writing.

# A. OWASP Top 10 / Core Web Vulnerabilities

## 1. SQL Injection -- blind vs error-based vs UNION-based. How do you confirm it without a working PoC payload visible?

SQL Injection occurs when untrusted input is concatenated directly into a SQL query instead of being passed as a parameterized value, allowing an attacker to alter the query's logic.

Error-based SQLi relies on the application returning verbose database error messages. The attacker deliberately sends a payload that breaks query syntax in a controlled way (e.g. injecting a type-conversion function around a subquery) so the resulting database error message itself leaks data such as the database version, table names, or even query results, because the error text contains the evaluated subquery output.

UNION-based SQLi appends an attacker-controlled SELECT statement to the original query using the UNION operator, so the extra rows appear directly in the normal page output -- provided the attacker first works out the correct number and data types of columns in the original query so the UNION aligns correctly.

Blind SQLi covers cases where no error message and no extra data appear in the response at all. It splits into two main techniques: Boolean-based blind, where the attacker injects a true/false condition and watches for any difference in the page's response (content, length, status code) to infer the answer one bit at a time; and Time-based blind, where the attacker injects a conditional delay function (such as SLEEP(5) or WAITFOR DELAY) and infers true/false based on how long the response takes to come back. A further variant, Out-of-Band SQLi, uses a completely separate channel -- typically forcing the database to make a DNS or HTTP request to an attacker-controlled listener -- when neither a visible difference nor a timing difference is usable.

When triaging a report that only includes a description or screenshot rather than a fully visible exploited payload, you confirm it by asking the researcher for the raw HTTP request/response pair (ideally exported straight from Burp), then either reproducing it yourself against a safe test account, or, if reproduction risks touching real data, checking for the telltale signs indirectly: response timing differences for time-based claims, or a database error string that's clearly leaking schema information for error-based claims. If reproduction genuinely cannot be done safely, request a screen recording showing the request being sent and the resulting behavior, rather than accepting a written claim alone.

## 2. Explain stored vs reflected vs DOM-based XSS. Which is most dangerous and why?

Reflected XSS occurs when the malicious payload is part of the request itself (commonly a URL parameter) and is echoed back immediately in the response without being stored anywhere -- it requires the victim to click a specifically crafted link.

Stored (persistent) XSS occurs when the payload is saved server-side, for example in a database field, comment, or user profile, and is then rendered to other users automatically whenever they view that stored content, with no need for them to click anything.

DOM-based XSS lives entirely on the client side: vulnerable JavaScript takes attacker-controlled data -- from the URL fragment, postMessage, document.referrer, or similar sources -- and unsafely writes it into the page DOM using sinks like innerHTML, document.write, or eval, without that data ever necessarily passing through the server at all. This matters because DOM XSS can bypass server-side input validation and even some WAFs entirely, since the payload may live only in the URL fragment (after the #), which browsers never send to the server.

Stored XSS is generally considered the most dangerous of the three because it requires zero interaction from the victim beyond simply visiting a normal page -- there's no crafted link to click, no social engineering step -- and it can compromise every visitor of that page automatically, giving it true worm potential (the Samy MySpace worm is the classic historical example). Reflected and DOM-based XSS both depend on getting a victim to click a specific link, which limits the realistic blast radius, even though DOM-based XSS in modern single-page applications can still be extremely serious.

## 3. CSRF -- what makes an app vulnerable, and what's the difference between CSRF and Clickjacking?

Cross-Site Request Forgery forces a victim's authenticated browser to silently submit a forged state-changing request to a target site, exploiting the fact that browsers automatically attach cookies to requests sent to a domain, regardless of which site triggered the request. An app becomes vulnerable when it relies solely on session cookies to authenticate state-changing requests without any additional, unguessable proof that the request was intentionally initiated by the user on that site -- typically a missing anti-CSRF token, no SameSite cookie attribute, and no validation of the Origin or Referer header.

Clickjacking is a different mechanism: the victim is tricked into clicking something that looks different from what they think they're clicking, typically by overlaying an invisible or disguised iframe of the real target site underneath a decoy interface. The click executes a genuine action on the embedded site -- such as toggling a privacy setting or liking a post -- but the victim believes they clicked something else entirely.

The key distinction is that CSRF forges a request with no genuine interaction by the victim with the real vulnerable page's interface at all -- the malicious page silently submits the forged request in the background. Clickjacking, by contrast, requires the victim to genuinely click on the real vulnerable page's actual UI; the attack is about disguising and hiding that real UI rather than forging a request without any interaction. CSRF is mitigated with tokens and SameSite cookies; clickjacking is mitigated with the X-Frame-Options header or a Content-Security-Policy frame-ancestors directive that prevents the page from being framed at all.

## 4. IDOR -- how do you distinguish a 'real' IDOR from a false positive during triage?

An Insecure Direct Object Reference happens when an application exposes a direct reference to an internal object -- an ID, key, or filename -- and fails to verify that the requester is actually authorized to access or modify that specific object, instead trusting the reference alone.

Confirming a genuine IDOR during triage involves several checks. First, the report needs to demonstrate two genuinely distinct authenticated identities, where account A successfully accesses or modifies account B's resource purely by changing an identifier, with no ownership or authorization check enforced server-side. Second, you need to confirm an actual sensitive data exposure or state change occurs -- a very common false positive is an endpoint that returns HTTP 200 regardless of authorization, but the data body returned is identical, empty, or already public, meaning nothing was actually leaked despite the misleading status code. Third, you need to rule out that the object is intentionally shared or public by design -- for example, a public profile endpoint meant to be viewable by anyone is not an IDOR just because the ID is sequential and enumerable. Finally, the severity hinges heavily on whether it's a read-only IDOR (information disclosure) or a write/delete IDOR (the ability to modify or destroy someone else's data), since these carry very different impact and should be scored very differently.

The most common false-positive pattern triage analysts run into is a report that shows a successful 200 OK after swapping an ID, without ever proving the actual returned data differs between the two accounts -- that gap is exactly what an analyst must independently verify before validating the report.

## 5. SSRF -- blind vs full-response SSRF, and how cloud metadata endpoints change severity.

Server-Side Request Forgery occurs when an application fetches a URL on the server's behalf based on attacker-controlled input -- common in webhook configurations, "import image from URL" features, or PDF/screenshot generation services -- allowing an attacker to make the server issue requests to destinations it shouldn't be able to reach, including internal-only infrastructure.

In full-response (in-band) SSRF, the content of the fetched URL's response is reflected directly back to the attacker, for example if an internal admin API's JSON response gets displayed in the application's UI after the attacker points the fetch feature at it -- making exploitation and data exfiltration trivial and immediately visible.

Blind SSRF occurs when the server does make the request, but the response content is never shown back to the attacker. Confirmation in this case relies on side channels: timing differences between requests to reachable versus unreachable internal hosts, or, most reliably, an out-of-band callback using a tool like Burp Collaborator or interactsh, where the attacker points the vulnerable parameter at their own listener domain and confirms the server actually made an outbound DNS or HTTP request, proving the SSRF exists even without seeing any response content.

Cloud metadata endpoints dramatically change severity because services like AWS, GCP, and Azure all expose an internal-only metadata service at the link-local address 169.254.169.254, which any instance can query without authentication to retrieve its own configuration, including, critically, temporary IAM role credentials. If SSRF allows an attacker to reach that endpoint, they can often extract live cloud credentials and use them to pivot into the broader cloud account: enumerating and reading storage buckets, calling other cloud APIs, or moving laterally to other resources. This turns what might otherwise be scored as a Medium internal-network-disclosure bug into a Critical full-cloud-account-compromise primitive, since the practical blast radius extends far beyond the single vulnerable application.

## 6. Broken Access Control vs Broken Authentication -- give a concrete example of each.

Broken Authentication covers failures in establishing who a user actually is -- weak password policies, missing rate-limiting on login attempts that enables credential stuffing, session fixation, predictable or insufficiently random session tokens, and password reset flows that allow account takeover. A concrete example: a password reset endpoint that issues a sequential, predictable token (such as an incrementing integer) instead of a cryptographically random one, letting an attacker simply guess or enumerate another user's reset token and take over their account.

Broken Access Control covers failures in enforcing what an already-authenticated user is permitted to do -- missing authorization checks, IDOR, privilege escalation, or forced browsing to administrative functionality. A concrete example: a regular authenticated user calling an admin-only API endpoint directly, and the server processes the request successfully because the admin check only existed in the frontend UI -- the button was hidden, but the backend never verified the caller's role.

The cleanest way to separate the two: authentication answers "who are you," while access control answers "what are you allowed to do, now that we know who you are." A system can have flawless authentication -- you are definitely, verifiably logged in as user A -- and still have completely broken access control, because nothing stops user A from performing actions reserved for an administrator.

## 7. Explain Mass Assignment and how it shows up in modern API-first apps.

Mass Assignment vulnerabilities occur when a framework automatically binds every field present in an incoming request body directly onto an internal data model or ORM object, without an explicit allow-list restricting which

fields a client is actually permitted to set.

The classic pattern: a "create user" or "update profile" endpoint is designed to accept only name and email from the client, but the underlying database model also contains other fields such as isAdmin or role, used internally. If the framework's binding logic blindly maps the entire incoming JSON body onto that model object, an attacker can simply add an extra "role":"admin" field to their request body, and the framework silently sets that field too, even though the legitimate UI never exposed it as an editable option.

This is especially common in API-first and GraphQL-driven applications, and in frameworks that auto-generate CRUD bindings as a convenience feature, precisely because the convenience of "just bind the whole request body to the object" trades away the safety of explicit field allow-listing. The fix is to always use explicit DTOs or allow-lists per endpoint defining exactly which fields a client may set, and to never deserialize an entire raw request body directly onto a model that contains privileged or internal-only fields.

## 8. What's the difference between a vulnerability that's 'exploitable' vs 'theoretical'? How does that affect your triage decision?

"Theoretical" generally means the underlying weakness genuinely exists in the code, but the real-world conditions needed to actually trigger meaningful impact either don't exist in this deployment (a feature is flagged off, the vulnerable path requires an internal-only network position with no demonstrated route to it, or it depends on a prerequisite, like an already-compromised account, that itself has no realistic path to compromise), or the maximum achievable impact is negligible even in the best case.

"Exploitable" means the researcher, or you during reproduction, can demonstrate an end-to-end, concrete unauthorized outcome that's reachable by a realistic attacker under the program's actual threat model -- for example, an external unauthenticated user, or a normal low-privilege authenticated user, depending on what the program defines as the relevant attacker.

This distinction directly drives triage outcomes because most bug bounty programs pay and credit for demonstrated impact, not merely for identifying a risky-looking code pattern. A report stating "this dependency version has a known CVE" without showing that the vulnerable code path is actually reachable and produces real impact in this specific application is typically triaged as Informative rather than a valid, rewarded finding. As an analyst, the standard question to ask is: "Can you demonstrate actual impact against this live target, not just identify that the pattern or outdated library is present?", and severity should only be assigned once that's been shown, not based on the theoretical worst case of the underlying weakness.

## 9. Open redirect -- why do some programs treat it as informative and others as a real bug?

An open redirect exists when an application takes a URL parameter and redirects the user to it without validating that the destination is internal or on an approved allow-list.

Many programs treat a standalone open redirect as Informative because, by itself, it doesn't directly expose any data or compromise any account -- the browser's address bar still shows the legitimate domain at the moment of the initial click, and while the victim does ultimately land on an attacker-controlled page, no information from the real site itself is leaked in that handoff.

Other programs treat the same finding as a real, higher-severity bug because of what it can be chained into. Open redirects make phishing links substantially more convincing, since a link starting with a trusted, well-known domain bypasses the instinctive suspicion users have toward obviously foreign URLs, and can even slip past some email security filters that allow-list trusted root domains. Far more seriously, an open redirect becomes genuinely dangerous when it can be chained into an OAuth or SSO flow -- manipulating a redirect parameter to redirect an authorization code or access token to an attacker-controlled endpoint can lead directly

to account takeover, turning a "low" issue into a critical one.

The practical triage takeaway is that severity should never be assigned to "open redirect" as a category in isolation -- the real question is always "redirect to accomplish what, specifically, within this application's architecture," since that context is what actually determines impact.

## 10. Business logic flaws -- these don't map cleanly to OWASP categories; how do you even look for them?

Business logic flaws are vulnerabilities in the application's intended workflow and rules, rather than in a specific technical injection point or memory-safety bug -- the code behaves exactly as written, but the underlying logic itself can be abused. Examples include race conditions in a checkout flow that let a single-use coupon be redeemed twice via parallel requests, submitting a negative quantity that turns a purchase into a refund, or skipping a required earlier step in a multi-step workflow by calling a later-stage API endpoint directly.

These don't map neatly onto a fixed OWASP Top 10 category because there's no single payload signature to scan for -- the closest bucket in OWASP's more recent Top 10 is "Insecure Design," which is intentionally broad and process-oriented rather than a specific injection class.

Finding them requires first genuinely understanding the intended end-to-end workflow, what the application's rules say should never be possible, and then deliberately testing against that understanding: attempting to skip steps in a sequential process, tampering with price, quantity, or currency parameters in ways the UI never allows but the backend might still accept, sending concurrent requests against the same single-use resource to look for race conditions, and reordering or replaying asynchronous callback or webhook steps out of their expected sequence. This is inherently manual, application-specific exploration rather than something an automated scanner can reliably find, which is exactly why strong business-logic testing is considered one of the more advanced, manual-first security testing disciplines.

## B. HackerOne-Specific Triage & Validation Concepts

### 1. Walk me through your mental checklist when a new report lands.

The order of these steps matters because each one gates the next, saving time by filtering out reports that don't need full effort.

First, a scope check: is the reported asset literally listed as in-scope in the program's policy? A report against an out-of-scope target is closed immediately regardless of how technically sound it is, because the customer has explicitly chosen not to accept risk responsibility for that asset.

Second, de-duplication: search prior reports and known issues for the same root cause on the same asset. A technically valid finding that's already been reported and is in remediation gets closed as a Duplicate, not because it's wrong, but because the customer is already aware, and credit belongs to the original reporter.

Third, reproduction: follow the reported steps exactly as written, ideally against a fresh test account, to confirm the claimed behavior actually occurs. If the steps are incomplete or ambiguous, this is the point to flag for more information rather than guess at what the researcher meant.

Fourth, severity scoring: once reproduced, determine the real-world impact and assign a CVSS vector reflecting confidentiality, integrity, and availability impact alongside attack complexity and privileges required.

Fifth, write the triage summary using a consistent structure: Issue Summary, Steps to Reproduce, Impact, Severity with vector, and Remediation, written so a developer who has never seen the original submission can act on it directly.

Finally, route the validated report to the customer's queue, or close it with a clear explanation if it didn't hold up.

### 2. What's an NMI (Needs More Info) and how do you decide when to send one vs reject outright?

NMI stands for "Needs More Info," a status used when a report can't yet be definitively judged valid or invalid because something essential is missing: incomplete reproduction steps, a proof-of-concept that doesn't actually demonstrate real impact, ambiguity about which account or environment was used, or missing evidence of the actual data that was exposed.

You send an NMI when the underlying technique described is plausible and the gap is purely about evidence or reproducibility -- for example, a researcher claims an IDOR but hasn't shown a second test account or before/after screenshots proving cross-account data exposure, and you genuinely cannot confirm it either way yet.

You reject outright, closing as Not Applicable or Informative, when, even assuming the report is reproduced exactly as described, it still wouldn't represent real security impact under the program's threat model -- for instance, self-XSS that requires the victim to paste a malicious payload into their own browser console, or behavior that turns out to be intended functionality.

A good triage analyst minimizes NMIs wherever reasonably possible. The researcher has usually spent significant time exploring the target and frequently understands the application's flow better than an analyst seeing it for the first time, so the better instinct is to first try reproducing with whatever has already been provided before reflexively asking for more -- excessive NMI requests frustrate skilled researchers and slow down the whole triage queue unnecessarily.

### 3. How does the same finding get a different severity in a dev/staging environment vs production?

The same technical finding can carry very different real-world severity depending on environment, because severity is fundamentally about realized impact, and impact depends on what's actually exposed in that specific environment.

A PII disclosure bug in a dev or staging environment populated only with synthetic or test data has near-zero real confidentiality impact, even though the underlying code path is identical to production -- there's simply no real person's data to leak, so it might reasonably be scored Low. The exact same bug in production, where the database contains genuine customer records, escalates dramatically: technically similar base vector, but the realistic impact assessment moves it to High or Critical, because real people's real data is now exposed.

The same logic applies to something like cache poisoning: poisoning a development cache that essentially no real user ever requests from has negligible real-world impact, while poisoning a production CDN or edge cache that serves live traffic can silently affect every visitor who happens to receive that poisoned cached response -- easily a High or Critical issue.

The practical triage takeaway is to always confirm exactly which environment the proof-of-concept was actually demonstrated against, and never assume severity transfers automatically between dev and production without separately validating reachability and impact in each, while also being mindful that a researcher testing directly against production sometimes raises its own scope and safe-harbor considerations worth checking against program rules.

## 4. How do you separate severity from urgency?

Severity is the relatively objective, largely environment-independent technical score of how bad a vulnerability is in principle. This is exactly what CVSS base metrics are designed to capture: how the vulnerability is reached, how complex exploitation is, what privileges are required, and how confidentiality, integrity, and availability are impacted if exploited successfully.

Urgency is how critical it actually is to fix this specific instance right now, for this specific customer, given real-world context that the CVSS base score deliberately does not capture: whether the asset is internet-facing or sits behind a VPN, whether it's currently being actively exploited in the wild, whether compensating controls like WAF rules or network segmentation meaningfully reduce practical exploitability today, whether public proof-of-concept code is already circulating, and what the customer's actual regulatory or business risk tolerance looks like.

A Critical CVSS-scored bug sitting behind multiple layers of network isolation with no internet exposure can legitimately be lower urgency than a Medium-scored bug that's fully internet-facing, trivially automatable, and currently being actively probed across many bug bounty programs. Frameworks like EPSS, which estimates the probability of exploitation activity in the next 30 days, and CISA's Known Exploited Vulnerabilities list exist specifically to separate "how bad is this in theory" from "how urgently does this need attention right now." A mature triage process treats severity and urgency as two distinct signals feeding into a prioritization decision, rather than collapsing everything into one number.

## 5. Roughly 60-80% of submitted reports turn out to be invalid -- how do you stay fast and consistent without becoming dismissive toward researchers?

The key is building a fast, repeatable mental funnel so the majority of invalid reports get filtered within the first couple of minutes: a scope check, followed by a quick sanity pass on whether the claimed vulnerability class is even plausible for this specific application, screening out recurring low-value patterns such as self-XSS claims, missing security headers reported as standalone critical findings without demonstrated impact, or descriptions of genuinely intended functionality.

Template responses are useful for the most common invalid patterns, informative findings like clickjacking on a page with no sensitive action, or version disclosure without exploitability, but every response should still be personalized with at least one sentence referencing the specific report's details, so the researcher knows a human actually reviewed their submission rather than receiving a generic copy-paste rejection.

Staying respectful matters regardless of processing volume, because a curt or dismissive tone toward a researcher who made an honest mistake damages trust in the entire program and discourages the kind of high-quality future submissions the bug bounty model depends on. The real discipline here is consistency under volume, not letting the two-hundredth report of the day receive a lower-effort review than the first one. Accuracy drift under fatigue is exactly what escalation and second-review processes exist to catch, but a strong analyst self-corrects for it rather than relying entirely on someone else catching the mistake later.

## 6. How would you handle a researcher who disagrees with your severity rating or pushes back aggressively on a 'Not Applicable' close?

First, re-verify your own reasoning before responding at all -- re-read the original report and your own triage note carefully to confirm nothing was actually missed, since a hasty, defensive reply tends to make things worse, while a slightly delayed but genuinely correct one rarely does.

Respond with the specific technical reasoning behind the decision, citing exactly which reproduction step failed, or precisely why the demonstrated impact falls short of the program's bar for that severity tier, rather than a vague restatement of general policy. Researchers who push back are very often doing so because the original close note wasn't specific enough for them to understand what exactly was wrong with their submission.

If the researcher provides new evidence or a stronger proof-of-concept in their pushback, the right move is to genuinely re-open and re-score the report. Credibility with the research community depends on analysts being correctable when presented with real new information, not merely being consistent for its own sake.

If, after a genuine re-review, the original decision still stands, explain that clearly and, where the program's process allows, offer to escalate to a second analyst or to the customer directly for a final call, rather than getting drawn into a prolonged back-and-forth. This protects both the researcher's sense of fairness in the process and your own time.

## 7. What's the difference between a duplicate report and a report that reveals new impact on an already-known issue -- and should the second one get partial credit?

A report is a true duplicate when it identifies exactly the same root cause, on the exact same vulnerable code path or endpoint, demonstrating the same or lesser impact as an already-existing report on file -- credit and reward go to whoever reported it first.

A second report deserves new or partial credit when it demonstrates meaningfully greater impact on top of an already-known issue. For example, if an original report showed only a low-impact information disclosure through a particular endpoint, and a second researcher demonstrates that the same endpoint can actually be chained into full account takeover, that second report is not a duplicate of the underlying root cause; it's genuinely new information about real-world impact that the original report never established.

The triage test to apply is: "if this second report didn't exist, would the customer still be missing actionable information needed to fully assess the real risk here?" If the answer is yes, the second report merits separate or bonus credit; if the customer already has everything they need to fully understand and remediate the issue from the first report alone, the second submission is a straight duplicate, regardless of how the researcher framed it.

# C. CVSS Scoring

## 1. Walk through CVSS 3.1 base metrics: AV, AC, PR, UI, S, C, I, A.

Attack Vector (AV) describes how the vulnerability is reached: Network (N) means it's reachable remotely over the internet; Adjacent (A) means the attacker needs to be on the same network segment, such as the same Wi-Fi or Bluetooth range; Local (L) means the attacker needs local access or a local login to the affected machine; Physical (P) means the attacker needs physical hands-on access to the device itself.

Attack Complexity (AC) describes whether exploitation is reliably achievable: Low (L) means there are no special conditions beyond normal attacker capability; High (H) means specific conditions outside the attacker's control are required, such as race condition timing or a particular victim configuration that must already exist.

Privileges Required (PR) describes what access level the attacker needs before exploiting the vulnerability: None (N) means a completely unauthenticated attacker can exploit it; Low (L) means a basic authenticated user account is sufficient; High (H) means the attacker already needs a privileged or admin-level account before the vulnerability can be exploited.

User Interaction (UI) describes whether a victim needs to take an action: None (N) means the exploit succeeds with zero action from any victim; Required (R) means a victim must do something, such as clicking a link or opening a file.

Scope (S) describes whether the impact stays confined to the vulnerable component: Unchanged (U) means the impact is limited to the security scope of the component that contains the vulnerability; Changed (C) means the impact extends into a different security authority entirely, such as a vulnerable web application component leading to compromise of the underlying host operating system.

Confidentiality, Integrity, and Availability Impact (C/I/A) are each rated None, Low, or High, describing respectively how much unauthorized data exposure, unauthorized data modification, and denial of legitimate access would result from successful exploitation.

These eight values combine into a vector string, for example AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N, which a CVSS calculator converts into a numeric base score from 0.0 to 10.0.

## 2. Score a sample vuln out loud: 'Reflected XSS in a low-traffic admin page requiring auth' -- what's AV, PR, UI here, and why?

Attack Vector is Network (N), because the page is still reached over standard HTTP/HTTPS even though it happens to be an admin-only page; network reachability, not traffic volume, is what AV measures.

Attack Complexity is Low (L), since a standard reflected XSS payload requires no unusual external conditions beyond crafting the malicious link.

Privileges Required reflects what the attacker themselves needs, not what the victim has: the attacker needs no privileges at all to craft and send the malicious link, so PR is typically None (N). What matters here instead is User Interaction, which is Required (R), because the victim, who happens to be an authenticated admin, must actually click the crafted link while logged in for the payload to execute in their authenticated session.

The interesting nuance in this scenario is that even though the attacker holds zero privileges, the realized impact can still be very high, because the victim's session is an admin session. If the resulting JavaScript can perform actions as that admin, creating new privileged accounts, changing security settings, or exfiltrating admin session tokens, then Confidentiality and Integrity impact should be scored High, not Low, purely because of who the victim is. The overall score, despite being "just XSS," can land in the Medium-to-High or even High range once that admin-level blast radius is properly accounted for; it should not be automatically downgraded

simply because the technique is reflected rather than stored.

### 3. What changed in CVSS 4.0 vs 3.1, and why does it matter for triage?

CVSS 4.0 removed the often-confusing single Scope metric from 3.1 and replaced it with clearly separated Vulnerable System impact metrics (VC, VI, VA) and Subsequent System impact metrics (SC, SI, SA), making it explicit which impacts hit the vulnerable component itself versus a different, downstream system.

It introduced Attack Requirements (AT) as a distinct metric separate from Attack Complexity, splitting "are there extra conditions needed for exploitation to even be possible" from "is the exploitation technique itself technically difficult."

It formalized Threat metrics, such as Exploit Maturity, and Environmental metrics as fully first-class, equally supported scoring dimensions, rather than the optional add-ons that most tooling and most analysts effectively ignored under 3.1, pushing the industry toward genuinely contextual, real-world scoring instead of base-score-only culture.

It also added Supplemental metrics, covering things like Safety impact, Automatable, and Recovery difficulty, which add richer risk-communication context without altering the core numeric score.

For triage work specifically, CVSS 4.0 reinforces and formalizes exactly the severity-versus-urgency distinction that matters in real prioritization: it pushes analysts toward presenting a fuller picture combining base, threat, and environmental context, rather than treating a single base score as the complete final word on how urgently something needs to be fixed.

### 4. CVSS includes threat and environmental metric groups that most programs underuse -- argue why a textbook-high CVSS score might still be low urgency for a given customer.

Consider a Critical CVSS base-scored remote code execution vulnerability in a legacy internal admin tool that is not internet-facing, sits behind VPN access plus MFA plus network segmentation, has no known public exploit code in circulation, and is already scheduled for full decommissioning next quarter.

The base score, something like AV:N/AC:L/PR:N/UI:N/C:H/I:H/A:H, correctly captures that, if this vulnerability were directly reachable, the consequences would be devastating. That part of the assessment doesn't change and shouldn't be argued away.

However, the Environmental metric group exists specifically to reflect that, in this customer's actual deployment, the effective attack vector is closer to Adjacent or Local once VPN and network segmentation are accounted for, since an external attacker simply cannot reach it directly. The Threat metric group, particularly Exploit Maturity, can reflect that there's no known exploit circulating and no evidence of active targeting. Together, these adjusted scores bring the realistic, contextual risk down substantially from the raw base score.

From a pure business-prioritization standpoint, a security team might quite reasonably choose to fix three Medium-base-scored, fully internet-facing vulnerabilities that are actively being probed by researchers across multiple bug bounty programs before getting to this one Critical-base-scored bug, and a strong triage analyst should be prepared to make exactly that argument rather than defaulting to "Critical numbers always go first," since blind base-score-ordering is a known, naive prioritization mistake.

### 5. How would you score something with high theoretical impact but very low exploitability (e.g. requires physical access or an already-compromised admin account)?

The right approach is to score both axes honestly rather than letting an impressive theoretical end-state inflate the access or complexity metrics.

If exploitation genuinely requires physical, hands-on access to the device, Attack Vector should be set to Physical (P), which caps the maximum possible base score well below what the identical impact would score if it were remotely reachable, even when Confidentiality, Integrity, and Availability impact are all rated High. The framework is deliberately designed this way, because how much access an attacker already needs is just as important to the overall risk picture as how bad the end result is.

If exploitation requires an already-compromised admin account as a prerequisite, Privileges Required should be set to High (PR:H), which similarly pulls the score down substantially, because CVSS is meant to reflect what an attacker needs to bring to the table, not just the impressive worst-case outcome.

There's also an important reasoning step beyond the score itself: don't double-count the prerequisite compromise as part of this specific report's contribution. The real question to ask is, "what can this vulnerability achieve beyond what a legitimate holder of that prerequisite access could already do anyway?" If a compromised low-privilege admin gains nothing beyond what their role already grants, the practical, reportable value of the finding is genuinely low regardless of how serious the theoretical impact sounds. But if it lets that compromised account escalate meaningfully beyond its intended privilege boundary, for instance reaching full root or infrastructure-wide access, then that marginal escalation, specifically, is the real severity the report should be scored against.

# D. Mobile (Android/iOS)

## 1. Insecure data storage on Android -- SharedPreferences, exported components, intent-based vulnerabilities.

SharedPreferences is Android's simple XML-based key-value store, commonly used for app settings. Many apps mistakenly store sensitive data, authentication tokens, PII, sometimes even passwords, here in plaintext. While modern Android blocks the old world-readable/world-writable file modes by default, on a rooted device, or in cases where the data should never have been stored this way at all, any other process with sufficient access can read this file directly from the app's private storage directory.

Exported components, Activities, Services, Broadcast Receivers, and Content Providers declared with android:exported="true" (or implicitly exported by having an intent-filter on older target SDK versions), can be invoked by any other application installed on the device, not just the legitimate app itself. If an exported component performs a sensitive action, such as transferring funds, exposing private data, or bypassing a login screen, without its own internal authorization check, any malicious app on the same device can trigger it directly through a crafted Intent. Conceptually, this is the same class of bug as a missing-authorization-check web vulnerability; it's just operating at the Android inter-process-communication layer instead of over HTTP.

Intent-based vulnerabilities arise when an app trusts data received through an Intent, Android's messaging object passed between components, without validating where that data actually came from or what it contains. If an app blindly acts on untrusted Intent data, such as a deep link parameter, a malicious app can craft and send a spoofed Intent to manipulate the target app's behavior, similar in spirit to trusting unvalidated input from an untrusted network source, just delivered through IPC instead.

## 2. What's an exported Activity/Service/Broadcast Receiver vulnerability, and how do you confirm exploitability without a rooted device?

An exported component vulnerability exists when an Activity, Service, or Broadcast Receiver marked as exported performs a sensitive operation without verifying that the caller is authorized to invoke it, meaning any other app installed on the same device, not only the legitimate app's own internal screens, can trigger that component directly.

You do not need root to confirm this. What you need is the target app installed normally on a test device or emulator, and the ability to send Intents directly to the exported component's exact component name, which can be done with no special privileges at all using the Android Debug Bridge. For example, an adb command can launch an exported Activity directly by component name, and a separate adb broadcast command can invoke an exported Broadcast Receiver directly, both running with normal, unprivileged app-level Activity Manager access, exactly as any other installed third-party app would be able to do, not with root.

If invoking the component this way produces an unauthorized state change, leaks data, or bypasses an authentication check that the component was supposed to enforce, that's a confirmed, non-theoretical exploit. Root is only genuinely needed for separate things, like directly reading another app's private filesystem storage off the device; it is not needed to trigger an exported IPC component, since exported components are, by their very definition, meant to be reachable from any other unprivileged app a user has installed. This distinction matters during triage: if a researcher claims "this requires root" for something that's actually reachable through a normal adb command or a simple unprivileged third-party app, the real-world attacker population is far larger than "users who specifically rooted their phone," and the severity should be raised accordingly.

## 3. Certificate pinning bypass -- conceptually, what does Frida/Objection let you do here, and how does that change your assessment of 'this requires jailbreak/root' claims?

Certificate pinning hardcodes the expected server certificate or public key directly inside the app, so even if a researcher installs their own custom CA certificate on the test device, the normal way to intercept and inspect HTTPS traffic through a proxy like Burp, the pinned app refuses to trust that custom certificate and blocks the connection, preventing standard traffic interception.

Frida is a dynamic instrumentation framework that lets a researcher hook and modify an app's code while it's actually running, without needing to decompile and permanently modify the installed APK. Objection is a runtime mobile exploration toolkit built directly on top of Frida that ships ready-made scripts specifically designed to disable SSL pinning. These scripts hook the app's own certificate validation functions at runtime and force them to always report "trusted," regardless of what certificate is actually presented, effectively neutralizing the pinning check from the inside.

This matters directly for how you assess a claim like "this requires a jailbroken or rooted device, so the severity should be lower." Frida-based pinning bypass commonly does not require root or jailbreak at all when using approaches like Frida Gadget injection into the app binary, and even in setups where rooting tools are used as the most convenient path, the real question for triage is whether the underlying vulnerability genuinely depends on a meaningfully high barrier, a privileged device compromise, or simply on network-traffic visibility, which any sufficiently equipped researcher or attacker can achieve regardless of root status, since pinning-bypass tooling is mature, widely documented, and easy to use. As a triage rule of thumb, "you'd need to bypass pinning to see this" is a weak justification on its own for downgrading severity.

## 4. iOS -- Keychain misuse, insecure IPC via URL schemes/universal links.

The iOS Keychain is the operating system's dedicated secure storage facility for credentials and tokens, and is generally far more secure by design than plaintext-style storage approaches on other platforms, but misuse still happens in practice. Common issues include storing Keychain items with an overly permissive accessibility attribute instead of a more restrictive option that ties access to the device being unlocked or to that specific device only; failing to require a device passcode or biometric check as part of accessing a particularly sensitive item; or storing highly sensitive, long-lived secrets in a way that allows them to be included in iCloud Keychain sync or device backups, when that data should arguably never leave the original device at all.

Insecure inter-process communication through custom URL schemes arises because apps can register their own custom scheme, allowing other apps or web pages to open them with parameters attached. If the receiving app trusts and directly acts on those parameters without validating where the call actually came from or sanitizing what it contains, for example navigating an in-app WebView straight to an attacker-supplied URL, or performing a sensitive action purely based on URL parameters with no further confirmation, then any other installed app or even a malicious web page can craft a link that triggers that behavior. This is conceptually the same class of issue as Android's exported-Intent problem, just expressed through the custom URL scheme handler instead.

Universal Links are Apple's more secure alternative, since they're cryptographically tied to a verified domain through a server-hosted association file, meaning only the legitimate domain owner can register and use that universal link, substantially reducing the risk of scheme-hijacking by other apps. However, the delivery mechanism being more secure does not automatically mean the receiving app validates what it does with the parameters once it receives them; an app can still mishandle data delivered through a perfectly legitimate universal link in exactly the same insecure way it might mishandle a custom URL scheme.

## E. Open Source Angle

### 1. How does triaging a report against an open-source project differ from triaging against a closed commercial SaaS product?

Source code visibility changes the validation process significantly. With an open-source target, you can pull up the actual vulnerable code directly and confirm root cause line by line, rather than purely inferring behavior from black-box HTTP responses. This generally makes technical validation faster and more conclusive, but it also raises the bar for what counts as a sufficient report, since "I believe this is vulnerable based on observed behavior" claims are now easy to either confirm or definitively disprove against the real source.

Already-known or already-patched issues are far more common with popular open-source projects, since they typically have public issue trackers, published security advisories, and CVE histories. A meaningfully higher proportion of "new discoveries" submitted to open-source programs turn out to already be publicly documented, already patched in a newer version, or duplicates of an existing CVE, so the deduplication step for open-source assets needs to explicitly include checking public vulnerability databases and the project's own advisory history, not only the program's internal report queue.

Maintainer bandwidth and expectations differ substantially too. Enterprise SaaS security teams typically have dedicated staff, defined response SLAs, and budget allocated for bounty payouts. Open-source maintainers are frequently volunteers or very small teams with limited available time, no dedicated security budget, and sometimes no formal incident response process at all. Triage summaries written for open-source programs therefore need to be more self-contained and immediately actionable, since you can't safely assume the same depth of in-house security expertise exists to translate "this is technically vulnerable" into "here is exactly what to change."

Scope nuance also carries more weight for open-source programs specifically: distinguishing whether a vulnerability lives in the project's own unmodified default behavior, versus arising from how one particular deployer configured or extended it, matters much more here than it typically does for a single closed SaaS product with one canonical deployment.

### 2. How would you distinguish a vulnerability that's specific to how a customer implemented an open-source library/framework vs a vulnerability in the framework itself?

A framework-level vulnerability exists in the open-source project's own default, unmodified code. If you stand up a completely clean install of the project with no custom configuration changes and the issue still reproduces, the vulnerability is framework-level. Fixing it requires a patch or new release from the upstream maintainers, and it potentially affects every single deployment of that framework worldwide, not just the one being currently triaged.

An implementation-specific vulnerability exists where the framework itself is secure by default, but a particular deployer or customer either misconfigured it, deliberately disabled a built-in security feature, wrote custom code on top of it that reintroduces a flaw the framework otherwise prevents, or is simply running an outdated, unpatched version when a fix already exists upstream. Examples include disabling a default CSRF-protection middleware, writing a custom authentication plugin with its own logic bug, or exposing a debug or administrative endpoint that ships disabled by default in a standard install.

This distinction matters enormously for triage because the two scenarios trigger entirely different response paths. A genuine framework-level finding typically needs to be escalated and credited to the upstream open-source project itself, potentially requiring a coordinated disclosure timeline, a CVE assignment, and an assessment of impact across every user of that framework globally. An implementation-specific finding, by contrast, is scoped and remediated entirely within that one customer's own environment, with no upstream

involvement needed at all. Misclassifying one as the other either falsely alarms an upstream maintainer about a problem that doesn't actually exist in their code, or under-reports a finding that genuinely should have triggered a much broader coordinated disclosure process.

The practical test to apply: reproduce the issue against a fresh, default-configured instance of the project with zero custom changes. If it reproduces as-is, it's framework-level. If you have to specifically enable or misconfigure something away from the secure-by-default state to trigger it, it's implementation-specific.

### 3. How do you write a triage summary that's technically accurate but still actionable for a small volunteer maintainer team vs a large enterprise security org?

For a small open-source maintainer team, the summary should stay tightly scoped and avoid assuming any dedicated security tooling or formal process exists on their end. Where the vulnerable file, function, or line is visible from the public source, state it explicitly. Provide the most minimal possible reproducible example, ideally a short code snippet or a single command, rather than a multi-step walkthrough that assumes familiarity with proxy tools like Burp. Where reasonable, suggest a concrete remediation direction directly, such as recommending parameterized queries at a specific point, or adding a specific allow-list check before a particular operation, rather than only describing the problem and leaving the fix entirely up to them, since the maintainers may not have in-house security expertise to translate "this is vulnerable" into "here's exactly what to change."

For a large enterprise security team, you can generally assume fluency with security tooling and frameworks, CVSS vectors, raw HTTP requests in Burp's format, internal ticketing references, and can focus more of the summary on business-impact framing: the scope of data exposure, any compliance implications, and which user segments are affected. They will typically route the actual code-level fix to an internal engineering team that needs deep technical detail, while the security team itself needs the risk framing to prioritize the finding correctly against their own existing backlog.

In both cases, the underlying structure stays the same: Issue Summary, Steps to Reproduce, Impact, and Remediation. What genuinely changes between the two audiences is the depth of hand-holding provided on the remediation step, and the framing used to convey urgency. Compliance and business-risk language tends to land better with an enterprise team, while straightforward technical urgency tends to be more effective with volunteer maintainers who have no compliance department driving their priorities.

### 4. How do you handle a report against an open-source dependency that's actually a known, already-disclosed CVE?

The first step is to verify whether the specific version and configuration actually reported is genuinely affected by the known CVE. Researchers sometimes misidentify an already-fixed version as still vulnerable, or the CVE in question may only apply under a specific, non-default configuration that isn't actually present here; matching a known CVE ID is not automatically the same as a correctly validated submission.

If the report is genuinely the same root cause as a known, publicly disclosed CVE, and the customer simply hasn't yet patched to a fixed version, this is usually still worth reporting to the customer, since they may not be aware they're running a vulnerable version. However, it's typically not eligible for the same researcher-discovery reward tier as a genuinely novel, previously unknown vulnerability, since the actual "discovery" effort here largely amounted to checking a version number against a public database rather than original security research. Most programs maintain specific bounty-table guidance distinguishing "known-CVE, outdated software" reports, often rewarded at a lower tier, from net-new vulnerability findings.

The deduplication process for open-source assets specifically needs to include checking sources like the National Vulnerability Database, GitHub Security Advisories, and the project's own published security

changelog for the exact version in use, not only the program's internal report history on HackerOne; for open-source targets, "already publicly known" is just as valid a reason to treat something as a duplicate as "already reported on this platform."

Finally, the triage summary should still flag the finding clearly to the customer, because a publicly known, already-disclosed CVE in active use is itself a high-urgency situation, public CVEs tend to get weaponized quickly once disclosed, while correctly noting in the internal classification that it represents a known, previously disclosed issue rather than a novel discovery, for accurate reward-tier assignment and program metrics.

# F. Report Writing & Communication

## 1. Practice writing a one-paragraph triage summary: Issue -> Steps to Reproduce -> Impact -> Recommended Fix.

A reliable template to default to every time: state the vulnerability type and location, then exactly how an attacker can trigger it, then the specific resulting impact and why the flaw exists, then the recommended fix. Keeping every summary in this exact order, even before adding any extra technical detail, means a developer who has never seen the original report can immediately understand what's wrong, how bad it is, and what to do about it, without needing to go back and read the raw hacker submission.

A worked example, for an IDOR finding: "Insecure Direct Object Reference in the invoice retrieval endpoint. By replacing the invoice ID parameter with another customer's invoice ID while authenticated as a different account, an attacker can view another customer's full invoice details, including billing address and partial card number, because the endpoint does not verify that the requested invoice actually belongs to the requesting account. Recommended fix: add a server-side ownership check comparing the invoice's associated account ID against the authenticated session's account ID before returning any data."

Notice that every sentence in that example does exactly one job: state the vulnerability class and location, state precisely how it's triggered, state precisely what's exposed and why the flaw exists, and state precisely what needs to change. Avoiding vague language like "could potentially expose sensitive information" in favor of stating exactly what data was actually confirmed exposed is what separates an actionable triage summary from a forwarded hacker report.

## 2. How do you write a rejection/NMI message to a researcher that's clear but doesn't damage the relationship or their HackerOne reputation unfairly?

The message should always state the specific, concrete reason for the decision rather than a generic boilerplate line, naming exactly which reproduction step couldn't be confirmed, exactly what evidence is missing, or exactly why the demonstrated impact doesn't meet the program's bar for a valid finding in this particular case. Vague rejections are precisely what trigger frustrated, drawn-out disputes, because the researcher genuinely doesn't know what to do differently next time.

It should also acknowledge the effort that went into the submission where genuine, even in a rejection. A brief, sincere line recognizing that the researcher correctly identified a real technical behavior, even though it doesn't rise to the level of a valid security impact under this program's scope, costs very little and meaningfully preserves the relationship.

Where the finding is closed as Not Applicable or Informative rather than genuinely Invalid, the message should make that distinction clear to the researcher, since these closure categories typically carry very different implications for their HackerOne reputation and statistics than an outright "invalid" or "spam" classification. Using the most accurate closure reason available, rather than defaulting to whichever closure feels fastest, directly protects a researcher's standing from an unfairly harsh classification of work that was actually reasonable, just ultimately not impactful enough.

Finally, if there's a reasonable path for the researcher to strengthen the submission, for instance, demonstrating actual data exposure rather than just a status code, the message should explicitly invite them to resubmit with that additional evidence, rather than closing the door outright on a report that might genuinely be valid with one more piece of proof.

### 3. How do you communicate a complex multi-step exploit chain to a customer engineer who isn't a security person?

Lead with the business impact first, not the technical mechanics. A single sentence describing what an attacker could ultimately achieve, for example, that an external attacker can fully take over any customer account without needing a password, gives the reader the right mental frame for why the following technical detail matters, before they've waded through any of it.

Follow that with a short, plain-language narrative of the overall chain, briefly defining any necessary jargon the first time it's used, rather than assuming familiarity with terms like SSRF, IDOR, or JWT. The goal at this stage is for a competent engineer with no security background to understand the overall shape of the attack: step one lets the attacker discover an internal address normally hidden from the outside, step two uses that to retrieve a temporary access credential, step three uses that credential to read another customer's data directly.

Only after that plain-language overview should the full technical detail and exact reproduction steps appear, ideally as a clearly separated supporting section or appendix, rather than being the very first thing the reader encounters. This lets a security-literate colleague who picks up the same report later dig into the raw technical detail without forcing every reader to start there.

Finally, attach concrete, specific remediation guidance to each individual step of the chain rather than only to the final outcome, since breaking even one early link in a multi-step exploit chain is often enough to fully neutralize the entire attack path, and engineers receiving the report need to know exactly which of their own components that responsibility actually falls on.

## End of Q&A Bank

Pair this with the Triage Practical worked example and the behavioral-question section in your earlier prep guide before the interview.